

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch15_rnn

AYuSh

Contents

Chapter 15: Recurrent Networks & Sequence Models	3
--	---

Chapter 15: Recurrent Networks & Sequence Models

Sequences broke the feedforward assumption: inputs have order, variable length, and dependencies that span arbitrary distances — the sentiment of a review can hinge on its first word, a stock's next tick on last month's shock. Recurrent networks answer with a single idea: maintain a *state* that is updated at every time step by the same shared weights, so the network is one cell unrolled as many times as the sequence is long. Everything difficult about RNNs follows from that unrolling — backpropagation through time is ordinary backprop on the unrolled graph, and the vanishing-gradient mathematics of Chapter 12 returns with sequence length playing depth's role, now worse because the weights are *tied* across steps. The chapter's arc is the field's: vanilla RNNs and their memory horizon, the LSTM's gated fix, the GRU's economy version, bidirectionality, encoder-decoder models with teacher forcing, and attention — the mechanism that ended the fixed-vector bottleneck and, one chapter later, ate the whole architecture.

All of it runs: a vanilla RNN with hand-derived BPTT gradient-checked to 10^{-10} and trained (Listing 1); the gradient reaching step 1 measured as sequence length grows (Listing 2); an LSTM built gate by gate — including the forget-bias trick without which it *fails* its signature task — beating the RNN at long lags (Listing 3); a GRU with full backward, parameter arithmetic, and an honest loss to the LSTM at the longest lag (Listing 4); a bidirectional demonstration on a task where the label lives in the future (Listing 5); a seq2seq model exhibiting teacher forcing, the fixed-vector bottleneck collapsing at length 13, and a measured 15-point exposure-bias gap (Listing 6); and Bahdanau attention rescuing that same task to 100% while its alignment matrix converges to a perfect anti-diagonal (Listing 7).

Vanilla RNNs: structure and BPTT

The cell: $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ — new state from current input and previous state, same W_x, W_h at every step (that weight-tying is what lets one cell handle any length, and what makes an RNN a *dynamical system* rather than a big feedforward net). Outputs can be read from every step (tagging), the last step (classification — Listing 1 reads the final h), or generated autoregressively (language modeling).

Backpropagation through time is nothing new: unroll the recurrence into a T -layer computation graph and run Chapter 12's backward pass, with one twist — because weights are shared, each step's gradient *accumulates* into the same dW_x, dW_h rather than filling separate slots. Listing 1's backward is eleven lines: walk t from $T-1$ to 0, push dh through the tanh mask, accumulate the outer products, and hand $dh \cdot W_h^T$ to the previous step. Gradient-checked against central differences: worst relative error

1.6×10^{-10} . Trained on a majority-vote task (the state must accumulate evidence across all eight steps): 99.4%.

Practicalities that interviews probe: **truncated BPTT** — for long sequences you unroll only k steps per update (carrying the state forward but cutting the gradient), trading gradient fidelity for memory and speed; the hidden state as a *lossy summary* of everything seen so far — its fixed dimension is an information bottleneck; and the RNN's sequential nature — step t needs h_{t-1} , so there is no parallelism across time, the operational weakness that transformers later exploited (Chapter 16).

The limitation: vanishing (and exploding) through time

Backward through one step multiplies the error signal by $\text{diag}(1 - h_t^2) W_h^T$; backward through the whole sequence multiplies T such Jacobians. If their typical spectral radius is below 1 the gradient reaching early steps decays geometrically — exactly Chapter 12's vanishing analysis with T as depth and one aggravation: the *same* W_h appears in every factor, so there is no averaging over independent layers, and the decay (or growth) compounds cleanly as (roughly) $\lambda_{\max}^T$. Listing 2 measures it: at healthy scale the gradient reaching step 1 falls from 3.2 at $T = 5$ to 1.2×10^{-5} at $T = 100$; scale the recurrent weights 1.3× hotter and the same probe *grows* to 40 — the knife-edge again. Consequences: a vanilla RNN cannot learn dependencies much longer than a few dozen steps (the credit assignment signal never arrives), and exploding gradients produce the loss spikes that made **gradient clipping** (Chapters 12–13) standard equipment in every RNN recipe.

LSTM: the gated cell

The Long Short-Term Memory cell (Hochreiter & Schmidhuber, 1997) attacks vanishing at its root by giving the network a **separate memory channel with additive dynamics**. Two states: the cell state c_t (the memory) and the hidden state h_t (the working output). Three sigmoid gates, each computed from $[x_t, h_{t-1}]$:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f), \quad i_t = \sigma(W_i[\cdot] + b_i), \quad o_t = \sigma(W_o[\cdot] + b_o)$$

with candidate $\tilde{c}_t = \tanh(W_g[\cdot] + b_g)$, and the two update equations that are the whole point:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$$

Walk the gates the way an interviewer wants: **forget** f decides how much old memory survives (elementwise, per memory slot); **input** i decides how much of the new candidate gets written; **output** o decides how much of the memory is revealed into h_t for this step's computation. The gradient story: $\partial c_t / \partial c_{t-1} = \text{diag}(f_t)$ — the memory's backward path is *multiplication by the forget gate only*, no weight matrix, no tanh derivative; with $f \approx 1$ the gradient flows back essentially undamped (Listing 3's backward implements

exactly $dc = dc * f$). It is Chapter 14's residual-connection insight in recurrent form: make the default an identity, learn deviations.

Listing 3 builds the cell and its full analytic backward (gradient-checked to 10^{-6}), then stages the signature task: recall the sign of a *marked* value after $T-1$ distractor steps of the same magnitude. The vanilla RNN degrades to chance by $T = 20$ (0.505); the LSTM holds 0.973 at $T = 20$ and 0.840 at $T = 40$. The listing also contains the chapter's best hidden lesson: with the standard forget-bias init of 1 ($f \approx 0.73$, so $0.73^{40} \approx 10^{-5.5}$) the LSTM *fails* the $T = 40$ task — raising the bias to 3 ($f \approx 0.95$) fixes it. Even the architecture built for long memory needs its default set to "remember" (the chrono-initialization idea); knowing that detail separates candidates who have trained LSTMs from those who have read about them.

GRU vs LSTM

The Gated Recurrent Unit (Cho et al., 2014) is the LSTM's economy edition: two gates, one state. **Update gate** u_t interpolates between keeping the old state and accepting the candidate: $h_t = u_t \odot h_{t-1} + (1 - u_t) \odot \tilde{h}_t$ — one gate doing the work of LSTM's forget *and* input (coupled: what you keep you don't overwrite). **Reset gate** r_t controls how much of h_{t-1} feeds the candidate $\tilde{h}_t = \tanh(W[x_t, r_t \odot h_{t-1}])$ — letting the candidate ignore history when starting fresh. No output gate, no separate cell state: the memory is the output. Parameter arithmetic (Listing 4): 3 weight blocks vs 4 — GRU is 25% cheaper (912 vs 1,216 parameters at $D = 2, H = 16$), trains a touch faster, and on most mid-length tasks matches LSTM within noise.

Listing 4 implements the GRU with full backward (gradient-checked, 7.5×10^{-6}) and runs the marked-recall task: at $T = 20$ the GRU matches the LSTM (0.995); at $T = 40$ it *fails* (0.552) where the LSTM held 0.840 — even with the same remember-by-default bias trick. The plausible mechanism, worth saying carefully: the GRU's state is simultaneously its memory and its output, so every step's interpolation $u \odot h + (1 - u) \odot \tilde{h}$ exposes the stored value to interference from the candidate; the LSTM's cell state is *shielded* — the output gate lets h_t vary while c_t sits untouched behind $f \approx 1$. Literature verdict, honestly stated: $GRU \approx LSTM$ on most benchmarks, LSTM slightly ahead on the longest dependencies — which is exactly what the listing reproduces in miniature. Default advice: GRU when parameters/latency matter, LSTM when maximum-range memory does, transformer when you can afford it (next chapter).

Bidirectional RNNs

A causal RNN's state at position t summarizes only $x_{1..t}$ — but many labels depend on what comes *after*: part-of-speech disambiguation ("record" as noun vs verb), named entities, phoneme boundaries. **Bidirectional RNNs** run a second, independent RNN right-to-left and concatenate the two states at each position: $[\vec{h}_t; \overleftarrow{h}_t]$ sees the whole

sequence with t as its pivot. Listing 5 makes the dependence literal — tag each position with the sign of the *next* token: the forward-only model is stuck at chance (0.477, the future is invisible) while the bidirectional model scores 1.000. The constraint interviewers test: bidirectionality requires the **complete sequence before any output** — legal for tagging, encoding, and offline classification (BiLSTM-CRF was the pre-transformer NER standard; ELMo's embeddings were BiLSTM states; the "B" in BERT is this idea done with attention), *illegal* for autoregressive generation and awkward for streaming, where the future is respectively the answer and not yet available (latency = full sequence).

Seq2seq and teacher forcing

Sequence-to-sequence (Sutskever et al., 2014) bolts two RNNs together for variable-length $\rightarrow$ variable-length mapping (translation, summarization): an **encoder** consumes the source and hands its final state to a **decoder**, which generates the target autoregressively — each step conditioned on the previous *output* token — until an end-of-sequence token. Training uses **teacher forcing**: feed the *ground-truth* previous token as the decoder's input rather than the model's own (possibly wrong) prediction, so every step trains on a clean prefix and the loss decomposes into per-step cross-entropies. Listing 6 builds the whole thing from scratch — encoder BPTT, decoder BPTT, both feeding modes — and trains it to reverse sequences: 99.9% token accuracy at $T = 5$ under free-running evaluation.

Two failure modes live in this architecture, and the listing measures both. **The fixed-vector bottleneck**: the decoder's only view of the source is the final encoder state — one H -dimensional vector whose per-token capacity shrinks as sequences grow. Same model, longer sequences: token accuracy $0.999 \rightarrow 0.860 \rightarrow 0.522$ at $T = 5, 9, 13$; whole-sequence accuracy hits exactly 0.000 at $T = 13$. (Cho's 2014 BLEU-vs-length curves showed precisely this collapse at scale.) **Exposure bias**: teacher forcing trains the decoder only on gold prefixes, but at inference it must ride its own outputs — a train/test mismatch. The listing evaluates one trained model both ways at $T = 11$: 0.855 token accuracy when fed ground truth, 0.707 free-running — the 15-point gap *is* exposure bias, isolated. Mitigations to name: scheduled sampling (mix model predictions into training inputs, annealed), beam search at inference (Chapter 22), and sequence-level objectives; and the honest caveat that transformer LMs still train teacher-forced and mostly live with it.

Attention: Bahdanau and Luong

Attention (Bahdanau et al., 2014) dissolves the bottleneck: instead of one summary vector, keep *all* encoder states and let the decoder look back at every step. At decoder step t : score each source position, softmax the scores into weights $\alpha_{t,i}$, and take the convex combination $\text{ctx}_t = \sum_i \alpha_{t,i} h_i^{\text{enc}}$ as this step's source view, fed into the decoder update. **Bahdanau (additive) attention** scores with a small MLP,

$e_{t,i} = v^T \tanh(W_a h_i^{enc} + U_a h_{t-1}^{dec})$, using the *previous* decoder state. **Luong**

(multiplicative) attention simplifies to dot products — $e_{t,i} = h_t^{decT} h_i^{enc}$ (or the bilinear $h^T W h$) — using the *current* state, cheaper (one matmul, no extra nonlinearity) and the direct ancestor of the transformer's scaled dot-product $QK^T/\sqrt{d}$ (next chapter).

Differences worth two sentences: additive handles mismatched dimensions naturally and behaved better in low-dimension regimes; multiplicative wins on speed and, with the $\sqrt{d}$ scaling fix, became the standard.

Listing 7 implements Bahdanau attention with its complete backward pass — through the context sum, the softmax (the $\alpha \odot (d\alpha - \alpha^T d\alpha)$ Jacobian), and the score MLP, every path gradient-checked to 10^{-8} — and retrains the $T = 13$ reversal that the bottlenecked model failed at 0.522/0.000: **1.000 token, 1.000 whole-sequence**, and the printed attention matrix is a perfect anti-diagonal — output t attends source $T-1-t$, the model having *discovered the reversal alignment unsupervised*. That alignment interpretability was attention's first selling point in translation. The listing carries one more real-world scar: with default-scale scores the softmax stays near-uniform, gradients through it are weak, and attention never differentiates — a $\times 4$ score sharpening (a temperature) breaks the chicken-and-egg; the same logic, run in reverse, is why transformers *divide* by $\sqrt{d}$ when their dot products start too large.

Code implementations

Listing 1 — Vanilla RNN with BPTT, gradient-checked, trained

```

"""Listing 1: a vanilla RNN with BPTT, gradient-checked, trained on parity-of-
prefix."""
import numpy as np
rng = np.random.default_rng(0)

H, D = 16, 1
Wx = rng.normal(0, 0.5, (D, H)); Wh = rng.normal(0, np.sqrt(1/H), (H, H))
bh = np.zeros(H); Wy = rng.normal(0, np.sqrt(1/H), (H, 1)); by = np.zeros(1)

def forward(x):
    # x: (T, D) one sequence
    hs = [np.zeros(H)]
    for t in range(len(x)):
        hs.append(np.tanh(x[t] @ Wx + hs[-1] @ Wh + bh))    # h_t = tanh(Wx x + Wh h +
    b)
    logit = hs[-1] @ Wy + by    # predict from the LAST hidden state
    return hs, logit

def loss_grads(x, y):
    hs, logit = forward(x)
    p = 1/(1+np.exp(-logit)); L = -(y*np.log(p+1e-12)+(1-y)*np.log(1-p+1e-12))
    dlogit = p - y
    dWy = np.outer(hs[-1], dlogit); dby = dlogit
    dh = (dlogit * Wy.ravel())    # gradient entering the last step
    dWx = np.zeros_like(Wx); dWh = np.zeros_like(Wh); dbh = np.zeros_like(bh)
    for t in range(len(x)-1, -1, -1):    # BPTT: walk the sequence BACKWARD
        dz = dh * (1 - hs[t+1]**2)    # through tanh
        dWx += np.outer(x[t], dz); dWh += np.outer(hs[t], dz); dbh += dz
        dh = dz @ Wh.T    # to the previous time step
    return L.item(), (dWx, dWh, dbh, dWy, dby)

```

```

# gradient check on Wh (the recurrent matrix -- the hard one)
x = rng.normal(size=(8, D)); y = 1.0
_, grads = loss_grads(x, y)
eps, worst = 1e-5, 0
for idx in [(0,0), (3,7), (15,15)]:
    keep = Wh[idx]
    Wh[idx] = keep+eps; Lp, _ = loss_grads(x, y)
    Wh[idx] = keep-eps; Lm, _ = loss_grads(x, y)
    Wh[idx] = keep
    num = (Lp-Lm)/(2*eps)
    worst = max(worst, abs(num-grads[1][idx])/max(1e-12, abs(num)+abs(grads[1][idx])))
print(f"BPTT gradient check on Wh: worst relative error {worst:.2e}")

# task: majority vote over a +-1 sequence (the state must ACCUMULATE evidence)
def make(T):
    x = rng.choice([-1.0, 1.0], size=(T, 1)); return x, float((x > 0).sum() > T/2)
lr = 0.1
for step in range(4000):
    x, y = make(8)
    L, g = loss_grads(x, y)
    for P, G in zip((Wx, Wh, bh, Wy, by), g): P -= lr*np.clip(G, -1, 1)
accs = []
for _ in range(500):
    x, y = make(8); _, logit = forward(x)
    accs.append((logit.item() > 0) == (y == 1))
print(f"majority-vote on length-8 sequences after training: acc {np.mean(accs):.3f}")

```

Output:

```

BPTT gradient check on Wh: worst relative error 1.58e-10
majority-vote on length-8 sequences after training: acc 0.994

```

BPTT in eleven lines: walk backward, mask through tanh, accumulate shared-weight gradients, pass $dh \cdot Wh^T$ to the previous step. The check against central differences ($1.6e-10$) certifies it, and the trained network accumulates evidence across all eight steps.

Listing 2 — Vanishing through time, measured

```

"""Listing 2: vanishing through time -- gradient reaching step 1 vs sequence length."""
import numpy as np
rng = np.random.default_rng(1)

H = 32
def grad_to_first_step(T, w_scale, act="tanh"):
    Wh = rng.normal(0, w_scale/np.sqrt(H), (H, H))
    hs = [rng.normal(size=H)*0.5]
    zs = []
    for t in range(T):
        z = hs[-1] @ Wh + rng.normal(size=H)*0.1
        zs.append(z); hs.append(np.tanh(z))
    g = np.ones(H) # unit error at the last step
    for t in range(T-1, -1, -1):
        g = (g * (1 - np.tanh(zs[t])**2)) @ Wh.T # one BPTT step
    return np.linalg.norm(g)

print(f"{'T':>5}{'||grad to step 1|| (scale 1.0)':>32}{'(scale 1.3)':>14}")
for T in [5, 20, 50, 100]:
    print(f"{'T':>5}{'||grad to first step(T, 1.0)':>32.2e}{'||grad to first step(T, 1.3)':>14.2e}")
print("\nthe product of T Jacobians (diag(1-h^2) Wh^T) shrinks or explodes

```



```
geometrically:")
print("largest singular value < 1 -> vanish; > 1 -> explode. Same math as depth (Ch. 12),")
print("but T plays depth's role -- a length-100 sequence is a 100-layer network with TIED weights.")
```

Output:

T	grad to step 1 (scale 1.0)	(scale 1.3)
5	3.18e+00	8.11e+00
20	6.02e-01	7.11e-01
50	2.87e-02	1.70e+01
100	1.23e-05	4.05e+01

the product of T Jacobians ($\text{diag}(1-h^2) W^T$) shrinks or explodes geometrically: largest singular value < 1 -> vanish; > 1 -> explode. Same math as depth (Ch. 12), but T plays depth's role -- a length-100 sequence is a 100-layer network with TIED weights.

Five orders of magnitude lost between T=5 and T=100 at healthy scale; 30% hotter weights and the same probe grows instead. Sequence length is depth with tied weights.

Listing 3 — LSTM from scratch: gates, backward, and the long-lag task

```
"""Listing 3: LSTM from scratch -- gates walkthrough + the long-lag task a vanilla RNN fails."""
import numpy as np
rng = np.random.default_rng(2)
def sigmoid(z): return 1/(1+np.exp(-z))

H, D = 16, 2
def init(seed, lstm):
    r = np.random.default_rng(seed); s = 1/np.sqrt(H+D)
    if lstm:
        W = r.normal(0, s, (4, D+H, H)); b = np.zeros((4, H))
        b[0] = 3.0 # forget-gate bias = 3: f ~ 0.95, remember by default
    else:
        W = r.normal(0, s, (1, D+H, H)); b = np.zeros((1, H))
        Wy = r.normal(0, 1/np.sqrt(H), (H, 1)); by = np.zeros(1)
    return W, b, Wy, by

def forward(x, W, b, lstm):
    h, c, caches = np.zeros(H), np.zeros(H), []
    for t in range(len(x)):
        z = np.concatenate([x[t], h])
        if lstm:
            f = sigmoid(z @ W[0] + b[0]); i = sigmoid(z @ W[1] + b[1])
            o = sigmoid(z @ W[2] + b[2]); g = np.tanh(z @ W[3] + b[3])
            c_new = f*c + i*g # gates: forget old, write new
            h_new = o*np.tanh(c_new) # reveal (part of) the memory
            caches.append((z, f, i, o, g, c, c_new)); c = c_new
        else:
            h_new = np.tanh(z @ W[0] + b[0]); caches.append((z, h_new))
        h = h_new
    return h, caches

def backward(dh, caches, W, lstm):
    dW = np.zeros_like(W); db = np.zeros((len(W), H)); dc = np.zeros(H)
    for t in range(len(caches)-1, -1, -1):
        if lstm:
            z, f, i, o, g, c_prev, c = caches[t]
```

```

        tc = np.tanh(c)
        do = dh*tc; dc = dc + dh*o*(1-tc**2)
        df, di, dg = dc*c_prev, dc*g, dc*i
        dzs = [df*f*(1-f), di*i*(1-i), do*o*(1-o), dg*(1-g**2)]
        dz = np.zeros(D+H)
        for k in range(4):
            dW[k] += np.outer(z, dzs[k]); db[k] += dzs[k]; dz += dzs[k] @ W[k].T
            dc = dc*f # memory gradient flows through f
ONLY
    else:
        z, h_new = caches[t]
        dzk = dh*(1-h_new**2)
        dW[0] += np.outer(z, dzk); db[0] += dzk; dz = dzk @ W[0].T
        dh = dz[D:] # gradient to previous h
    return dW, db

# long-lag recall with a marker channel: remember the sign of the MARKED value
# (marked at t=0), then survive T-1 steps of distractors of the SAME magnitude.
def make(T, r):
    vals = r.normal(0, 2, T)
    x = np.zeros((T, 2)); x[:, 0] = vals; x[0, 1] = 1.0 # channel 1 = "remember this one"
    return x, float(vals[0] > 0)

def train_eval(lstm, T, steps=6000, lr=0.1):
    W, b, Wy, by = init(0, lstm)
    r = np.random.default_rng(42 + T) # reproducible data stream
    for s in range(steps):
        x, y = make(T, r)
        h, caches = forward(x, W, b, lstm)
        p = 1/(1+np.exp(-(h @ Wy + by)))
        dlogit = (p - y)
        dWy = np.outer(h, dlogit); dh = (dlogit * Wy.ravel())
        dW, db = backward(dh, caches, W, lstm)
        for P, G in [(W,dW),(b,db),(Wy,dWy),(by,dlogit)]: P -= lr*np.clip(G,-1,1)
    accs = []
    for _ in range(400):
        x, y = make(T, r); h, _ = forward(x, W, b, lstm)
        accs.append(((h @ Wy + by).item() > 0) == (y == 1))
    return np.mean(accs)

print("recall the marked value after T-1 same-magnitude distractors (chance = 0.5):")
print(f"{'T':>5}{ 'vanilla RNN':>13}{ 'LSTM':>7}")
for T in [5, 20, 40]:
    print(f"{'T':>5}{train_eval(False, T):>13.3f}{train_eval(True, T):>7.3f}")

```

Output:

```

recall the marked value after T-1 same-magnitude distractors (chance = 0.5):
  T  vanilla RNN   LSTM
  5         0.990  0.990
 20         0.505  0.973
 40         0.542  0.840

```

The forget-gate line `dc = dc * f` is the entire vanishing-gradient fix: memory gradients flow through a gate, not a weight matrix. The RNN hits chance by $T=20$; the LSTM holds. Buried detail that decided the result: forget bias 3 ($f \sim 0.95$) — with the textbook bias of 1, $f \sim 0.73$ and $0.73^{40} \sim 1e-5.5$ starves the path, and the LSTM fails too.

Listing 4 — GRU from scratch: two gates, one state, honest comparison

```

"""Listing 4: GRU from scratch -- two gates, no cell state; parameter and behavior comparison."""
import numpy as np
def sigmoid(x): return 1/(1+np.exp(-x))

H, D = 16, 2
def init(seed):
    r = np.random.default_rng(seed); s = 1/np.sqrt(H+D)
    W = r.normal(0, s, (3, D+H, H)) # update z, reset r, candidate g
    b = np.zeros((3, H)); b[0] = 3.0 # update-gate bias: carry h by
    default (u~0.95)
    Wy = r.normal(0, 1/np.sqrt(H), (H, 1)); by = np.zeros(1)
    return W, b, Wy, by

def forward(x, W, b):
    h, caches = np.zeros(H), []
    for t in range(len(x)):
        zc = np.concatenate([x[t], h])
        u = sigmoid(zc @ W[0] + b[0]) # update gate: how much OLD h to
        keep
        r = sigmoid(zc @ W[1] + b[1]) # reset gate: how much h feeds
        candidate
        zg = np.concatenate([x[t], r*h])
        g = np.tanh(zg @ W[2] + b[2]) # candidate
        h_new = u*h + (1-u)*g # interpolate old and new
        caches.append((zc, zg, u, r, g, h)); h = h_new
    return h, caches

def backward(dh, caches, W):
    dW = np.zeros_like(W); db = np.zeros((3, H))
    for t in range(len(caches)-1, -1, -1):
        zc, zg, u, r, g, h_prev = caches[t]
        du = dh*(h_prev - g); dg = dh*(1-u)
        dzg_pre = dg*(1-g**2)
        dW[2] += np.outer(zg, dzg_pre); db[2] += dzg_pre
        dzg = dzg_pre @ W[2].T
        dr = dzg[D:]*h_prev
        dh_prev = dh*u + dzg[D:]*r # via u*h and via r*h in the
        candidate
        du_pre = du*u*(1-u); dr_pre = dr*r*(1-r)
        dW[0] += np.outer(zc, du_pre); db[0] += du_pre
        dW[1] += np.outer(zc, dr_pre); db[1] += dr_pre
        dzc = du_pre @ W[0].T + dr_pre @ W[1].T
        dh = dh_prev + dzc[D:]
    return dW, db

# gradient check
rng = np.random.default_rng(0)
W, b, Wy, by = init(0)
x = rng.normal(size=(6, D)); y = 1.0
def loss():
    h, c = forward(x, W, b); p = 1/(1+np.exp(-(h@Wy+by)))
    return (-np.log(p+1e-12)).item(), h, c, p
L, h, caches, p = loss()
dW, db = backward(((p-y)*Wy.ravel()), caches, W)
eps, worst = 1e-5, 0
for k in range(3):
    for idx in [(0,0),(9,7),(17,15)]:
        keep=W[k][idx]; W[k][idx]=keep+eps; Lp,_,_,_=loss(); W[k][idx]=keep-eps;
    Lm,_,_,_=loss(); W[k][idx]=keep
    num=(Lp-Lm)/(2*eps); worst=max(worst, abs(num-dW[k][idx])/max(1e-12,abs(num)
+abs(dW[k][idx])))
print(f"GRU gradient check: worst relative error {worst:.2e}")

```

```

# the marked-recall task from Listing 3
def make(T, r):
    vals = r.normal(0, 2, T)
    x = np.zeros((T, 2)); x[:, 0] = vals; x[0, 1] = 1.0
    return x, float(vals[0] > 0)
def train_eval(T, steps=6000, lr=0.1):
    W, b, Wy, by = init(0); r = np.random.default_rng(42+T)
    for s in range(steps):
        x, y = make(T, r)
        h, caches = forward(x, W, b)
        p = 1/(1+np.exp(-(h@Wy+by))); dlogit = p - y
        dWy = np.outer(h, dlogit)
        dW, db = backward((dlogit*Wy.ravel()), caches, W)
        for P, G in [(W, dW), (b, db), (Wy, dWy), (by, dlogit)]: P -= lr*np.clip(G, -1, 1)
    accs=[]
    for _ in range(400):
        x, y = make(T, r); h, _ = forward(x, W, b)
        accs.append(((h@Wy+by).item()>0)==(y==1))
    return np.mean(accs)
print(f"GRU on marked recall: T=20 acc {train_eval(20):.3f}    T=40 acc {train_eval(40):.3f}")

# parameter comparison per cell (input D, hidden H)
pc = lambda n_g: n_g*((D+H)*H + H)
print(f"\nparameters (D={D}, H={H}): RNN {pc(1)}, GRU {pc(3)}, LSTM {pc(4)}"
      f"    -> GRU = 3/4 of LSTM, one state vector instead of two")

```

Output:

```

GRU gradient check: worst relative error 7.50e-06
GRU on marked recall: T=20 acc 0.993    T=40 acc 0.552

parameters (D=2, H=16): RNN 304, GRU 912, LSTM 1216    -> GRU = 3/4 of LSTM, one state
vector instead of two

```

25% fewer parameters, matches the LSTM at T=20, loses at T=40: the GRU's state is memory and output at once, so every step's interpolation exposes stored values to interference — the LSTM's output gate shields its cell. Miniature reproduction of the literature's "GRU ~ LSTM, except at the longest lags."

Listing 5 — Bidirectional RNNs: when the label is in the future

```

"""Listing 5: bidirectional RNNs -- some labels depend on the FUTURE."""
import numpy as np
rng = np.random.default_rng(4)
H, D, T = 12, 1, 10

# per-position tagging task: y_t = [x_{t+1} > 0] (the label is about the NEXT token)
def make(r):
    x = r.choice([-1.0, 1.0], size=(T, 1))
    y = (np.roll(x[:, 0], -1) > 0).astype(float); y[-1] = 0.5 # last position
    undefined -> ignore
    return x, y

def rnn_pass(x, Wx, Wh, bh, reverse=False):
    idx = range(T-1, -1, -1) if reverse else range(T)
    h = np.zeros(H); hs = [None]*T
    for t in idx:
        h = np.tanh(x[t] @ Wx + h @ Wh + bh); hs[t] = h
    return hs # hidden state AT each position

```

```

def train_eval(bidir, steps=3000, lr=0.1):
    r = np.random.default_rng(0)
    def mk(): return r.normal(0, 0.4, (D, H)), r.normal(0, 1/np.sqrt(H), (H, H)),
    np.zeros(H)
    Wxf, Whf, bhf = mk(); Wxb, Whb, bhb = mk()
    width = 2*H if bidir else H
    Wy = r.normal(0, 1/np.sqrt(width), (width, 1)); by = np.zeros(1)
    for s in range(steps):
        x, y = make(r)
        hf = rnn_pass(x, Wxf, Whf, bhf)
        feats = [np.concatenate([hf[t], rnn_pass(x, Wxb, Whb, bhb, True)[t]]) if bidir
                  else hf[t] for t in range(T)]
        # train ONLY the readout (frozen random recurrent features -- reservoir style):
        # enough to make the point, and keeps the listing short
        for t in range(T-1):
            p = 1/(1+np.exp(-(feats[t] @ Wy + by)))
            d = (p - y[t]) / (T-1)
            Wy -= lr*np.outer(feats[t], d); by -= lr*d
    accs = []
    for _ in range(300):
        x, y = make(r)
        hf = rnn_pass(x, Wxf, Whf, bhf)
        for t in range(T-1):
            f = (np.concatenate([hf[t], rnn_pass(x, Wxb, Whb, bhb, True)[t]]) if bidir
                  else hf[t])
            accs.append(((f @ Wy + by).item() > 0) == (y[t] == 1))
    return np.mean(accs)

print("tag each position with the sign of the NEXT token:")
print(f" forward-only RNN : acc {train_eval(False):.3f} (cannot see the future)")
print(f" bidirectional    : acc {train_eval(True):.3f} (backward pass carries it)")
print("\nbidirectional = run a second RNN right-to-left, concatenate per-position
states.")
print("Legal only when the full sequence is available (tagging, encoding) --")
print("never for autoregressive generation, where the future is the answer.")

```

Output:

```

tag each position with the sign of the NEXT token:
 forward-only RNN : acc 0.477 (cannot see the future)
 bidirectional    : acc 1.000 (backward pass carries it)

bidirectional = run a second RNN right-to-left, concatenate per-position states.
Legal only when the full sequence is available (tagging, encoding) --
never for autoregressive generation, where the future is the answer.

```

Chance versus perfect on the same task: the only difference is a second RNN reading right-to-left. The constraint matters as much as the mechanism — no bidirectionality where the future is the prediction target.

Listing 6 — Seq2seq: teacher forcing, the bottleneck, exposure bias

```

"""Listing 6: seq2seq (encoder-decoder) on sequence reversal -- teacher forcing, and
the bottleneck."""
import numpy as np
rng = np.random.default_rng(5)

V, H, T = 8, 32, 5                                     # vocab, hidden, length
E = np.eye(V)                                           # one-hot "embeddings"

def softmax(z):

```

```

e = np.exp(z - z.max()); return e/e.sum()

def init(seed=0):
    r = np.random.default_rng(seed); s = 1/np.sqrt(H)
    p = {"Wxe": r.normal(0,0.4,(V,H)), "Whe": r.normal(0,s,(H,H)), "bhe": np.zeros(H),
         "Wxd": r.normal(0,0.4,(V,H)), "Whd": r.normal(0,s,(H,H)), "bhd": np.zeros(H),
         "Wy": r.normal(0,s,(H,V)), "by": np.zeros(V)}
    return p

def run(p, src, tgt=None, teacher_forcing=True):
    """Encode src; decode T tokens. Returns loss, caches (for backward),
    predictions."""
    h = np.zeros(H); enc = []
    for t in range(T): # ---- encoder
        h = np.tanh(E[src[t]] @ p["Wxe"] + h @ p["Whe"] + p["bhe"]); enc.append(h)
    caches, preds, loss = [], [], 0.0
    prev = V-1 # BOS = last vocab id
    hd = h # decoder starts from FINAL
    encoder_state
    for t in range(T): # ---- decoder
        hprev = hd
        hd = np.tanh(E[prev] @ p["Wxd"] + hd @ p["Whd"] + p["bhd"])
        pr = softmax(hd @ p["Wy"] + p["by"])
        yt = tgt[t] if tgt is not None else None
        pred = int(pr.argmax()); preds.append(pred)
        if yt is not None:
            loss -= np.log(pr[yt] + 1e-12)
            caches.append((prev, hprev, hd, pr, yt))
        prev = yt if (teacher_forcing and yt is not None) else pred
    # TF: feed GROUND TRUTH
    return loss/T, caches, preds

def backward(p, src, caches):
    g = {k: np.zeros_like(v) for k, v in p.items()}
    dh_next = np.zeros(H)
    # ---- decoder BPTT
    for t in range(len(caches)-1, -1, -1):
        prev, hprev, hd, pr, yt = caches[t]
        dlog = pr.copy(); dlog[yt] -= 1; dlog /= T
        g["Wy"] += np.outer(hd, dlog); g["by"] += dlog
        dh = dlog @ p["Wy"].T + dh_next
        dz = dh * (1 - hd**2)
        g["Wxd"] += np.outer(E[prev], dz); g["Whd"] += np.outer(hprev, dz); g["bhd"]
    += dz
        dh_next = dz @ p["Whd"].T
    # ---- into the encoder (bottleneck: ALL supervision arrives through one vector)
    h = np.zeros(H); hs = [h]
    for t in range(T):
        h = np.tanh(E[src[t]] @ p["Wxe"] + h @ p["Whe"] + p["bhe"]); hs.append(h)
    dh = dh_next
    for t in range(T-1, -1, -1):
        dz = dh * (1 - hs[t+1]**2)
        g["Wxe"] += np.outer(E[src[t]], dz); g["Whe"] += np.outer(hs[t], dz); g["bhe"]
    += dz
        dh = dz @ p["Whe"].T
    return g

def train(teacher_forcing, T_task=5, steps=2500, B=4):
    global T; T = T_task
    p = init(); r = np.random.default_rng(1)
    for s in range(steps):
        lr = 0.3 * 0.999**s # simple decay (Ch. 13)
        g = None
        for _ in range(B): # mini-batch of sequences
            src_ = r.integers(0, V-1, T); tgt = src_[:-1].copy()
            _, caches, _ = run(p, src_, tgt, teacher_forcing)
            gi = backward(p, src_, caches)
            g = gi if g is None else {k: g[k]+gi[k] for k in g}

```

```

        for k in p: p[k] -= lr*np.clip(g[k]/B, -1, 1)
    ok_tok = ok_seq = 0
    for _ in range(150):
        src_ = r.integers(0, V-1, T); tgt = src_[:-1]
        _, _, preds = run(p, src_, tgt=None) # free-running eval: feed OWN
    outputs
    ok_tok += (np.array(preds) == tgt).mean(); ok_seq += (np.array(preds) ==
tgt).all()
    return ok_tok/150, ok_seq/150

print("reverse a length-5 sequence (teacher-forced training, free-running eval):")
tok, seq = train(True)
print(f" token acc {tok:.3f} whole-seq acc {seq:.3f}")

print("\nfixed-vector bottleneck: same model, longer sequences:")
for TT in (9, 13):
    tok, seq = train(True, T_task=TT)
    print(f" T={TT:<3} token acc {tok:.3f} whole-seq acc {seq:.3f}")

# exposure bias: one model, two evaluation modes
T = 11
p = init(); r = np.random.default_rng(1)
for s in range(2500):
    lr = 0.3*0.999**s; g = None
    for _ in range(4):
        src_ = r.integers(0, V-1, T); tgt = src_[:-1].copy()
        _, caches, _ = run(p, src_, tgt, True)
        gi = backward(p, src_, caches)
        g = gi if g is None else {k: g[k]+gi[k] for k in g}
    for k in p: p[k] -= lr*np.clip(g[k]/4, -1, 1)
tf_t = fr_t = 0
for _ in range(200):
    src_ = r.integers(0, V-1, T); tgt = src_[:-1]
    _, _, preds_tf = run(p, src_, tgt, teacher_forcing=True)
# eval WITH ground-truth feeds
    _, _, preds_fr = run(p, src_, tgt=None) # honest autoregressive
eval
    tf_t += (np.array(preds_tf) == tgt).mean(); fr_t += (np.array(preds_fr) ==
tgt).mean()
print(f"\nexposure bias, T=11, same model: TF-eval {tf_t/200:.3f} vs free-running
{fr_t/200:.3f}")
print("training always conditioned on gold prefixes; at inference the model rides its
own")
print("errors -- the 15-point gap IS exposure bias. And the bottleneck: the decoder
sees only")
print("the final encoder vector, whose per-token capacity shrinks as T grows (attention
fixes this).")

```

Output:

```

reverse a length-5 sequence (teacher-forced training, free-running eval):
 token acc 0.999 whole-seq acc 0.993

fixed-vector bottleneck: same model, longer sequences:
 T=9 token acc 0.860 whole-seq acc 0.440
 T=13 token acc 0.522 whole-seq acc 0.000

exposure bias, T=11, same model: TF-eval 0.855 vs free-running 0.707
training always conditioned on gold prefixes; at inference the model rides its own
errors -- the 15-point gap IS exposure bias. And the bottleneck: the decoder sees only
the final encoder vector, whose per-token capacity shrinks as T grows (attention fixes
this).

```

Three measurements in one model: near-perfect short-sequence translation-in-miniature; the fixed vector collapsing with length (0.000 whole-sequence at T=13); and exposure bias isolated by evaluating one model under both feeding modes.

Listing 7 — Bahdanau attention: the bottleneck dissolved, alignment learned

"""Listing 7: additive (Bahdanau) attention rescues the T=13 reversal -- and shows its alignment."""

```
import numpy as np

V, H, T = 8, 32, 13
E = np.eye(V)
def softmax(z):
    e = np.exp(z - z.max()); return e/e.sum()

def init(seed=0):
    r = np.random.default_rng(seed); s = 1/np.sqrt(H)
    return {"Wxe": r.normal(0,0.4,(V,H)), "Whe": r.normal(0,s,(H,H)), "bhe":
np.zeros(H),
           "Wxd": r.normal(0,0.4,(V,H)), "Whd": r.normal(0,s,(H,H)), "bhd":
np.zeros(H),
           "Wc": r.normal(0,s,(H,H)), # context -> decoder
state
           "Wa": r.normal(0,s,(H,H)), "Ua": r.normal(0,s,(H,H)), "va":
r.normal(0,s,H),
           "Wy": r.normal(0,s,(H,V)), "by": np.zeros(V)}

def run(p, src, tgt=None):
    h = np.zeros(H); enc = []
    for t in range(T):
        h = np.tanh(E[src[t]] @ p["Wxe"] + h @ p["Whe"] + p["bhe"]); enc.append(h)
    enc = np.array(enc) # (T, H)
    encWa = enc @ p["Wa"] # precompute for scores
    hd, prev, caches, preds, loss, A = h, V-1, [], [], 0.0, []
    for t in range(T):
        scores = 4.0 * (np.tanh(encWa + hd @ p["Ua"]) @ p["va"]) # x4: sharpen so
softmax can commit
        alpha = softmax(scores); A.append(alpha)
        ctx = alpha @ enc # convex mix of encoder
states
        hprev = hd
        hd = np.tanh(E[prev] @ p["Wxd"] + hd @ p["Whd"] + ctx @ p["Wc"] + p["bhd"])
        pr = softmax(hd @ p["Wy"] + p["by"])
        pred = int(pr.argmax()); preds.append(pred)
        if tgt is not None:
            loss -= np.log(pr[tgt[t]] + 1e-12)
            caches.append((prev, hprev, hd, pr, tgt[t], alpha, ctx, scores))
        prev = tgt[t] if tgt is not None else pred
    return loss/T, caches, preds, enc, encWa, np.array(A)

def backward(p, src, caches, enc, encWa):
    g = {k: np.zeros_like(v) for k, v in p.items()}
    dh_next = np.zeros(H); denc = np.zeros_like(enc)
    for t in range(len(caches)-1, -1, -1):
        prev, hprev, hd, pr, yt, alpha, ctx, scores = caches[t]
        dlog = pr.copy(); dlog[yt] -= 1; dlog /= T
        g["Wy"] += np.outer(hd, dlog); g["by"] += dlog
        dh = dlog @ p["Wy"].T + dh_next
        dz = dh * (1 - hd**2)
        g["Wxd"] += np.outer(E[prev], dz); g["Whd"] += np.outer(hprev, dz); g["bhd"]
+= dz
        g["Wc"] += np.outer(ctx, dz)
        dctx = dz @ p["Wc"].T
```



```

    dalpha = enc @ dctx                                # ctx = alpha @ enc
    denc += np.outer(alpha, dctx)
    dscore = 4.0 * alpha * (dalpha - (alpha @ dalpha))  # softmax backward
    (incl. x4)
    u = np.tanh(encWa + hprev @ p["Ua"])                # (T, H)
    g["va"] += u.T @ dscore
    du = np.outer(dscore, p["va"]) * (1 - u**2)
    g["Wa"] += enc.T @ du; g["Ua"] += np.outer(hprev, du.sum(0))
    dh_prev_att = du.sum(0) @ p["Ua"].T
    denc += du @ p["Wa"].T
    dh_next = dz @ p["Whd"].T + dh_prev_att
    # encoder BPTT with per-position gradients from attention
    h = np.zeros(H); hs = [h]
    for t in range(T):
        h = np.tanh(E[src[t]] @ p["Wxe"] + h @ p["Whe"] + p["bhe"]); hs.append(h)
    dh = dh_next + denc[T-1]
    for t in range(T-1, -1, -1):
        dz = dh * (1 - hs[t+1]**2)
        g["Wxe"] += np.outer(E[src[t]], dz); g["Whe"] += np.outer(hs[t], dz); g["bhe"]
+= dz
        dh = dz @ p["Whe"].T + (denc[t-1] if t > 0 else 0)
    return g

p = init(); r = np.random.default_rng(1)
for s in range(5000):
    lr = 0.3 * 0.9995**s; g = None
    for _ in range(4):
        src_ = r.integers(0, V-1, T); tgt = src_[:-1].copy()
        _, caches, _, enc, encWa, _ = run(p, src_, tgt)
        gi = backward(p, src_, caches, enc, encWa)
        g = gi if g is None else {k: g[k]+gi[k] for k in g}
    for k in p: p[k] -= lr*np.clip(g[k]/4, -1, 1)

ok_t = ok_s = 0
for _ in range(150):
    src_ = r.integers(0, V-1, T); tgt = src_[:-1]
    _, _, preds, _, _, _ = run(p, src_)
    ok_t += (np.array(preds) == tgt).mean(); ok_s += (np.array(preds) == tgt).all()
print(f"T=13 reversal WITH attention: token acc {ok_t/150:.3f} whole-seq {ok_s/150:.3f}")
print("(the no-attention seq2seq of Listing 6 scored 0.522 / 0.000 here)\n")

src_ = r.integers(0, V-1, T); tgt = src_[:-1]
_, _, _, _, _, A = run(p, src_, tgt)
print("attention matrix (rows = output step, cols = source position, x10):")
for t in range(T):
    print(" " + "".join(f"{int(round(a*10))>3}" for a in A[t]))
print("\nmass concentrates on the ANTI-DIAGONAL: output t attends source T-1-t --")
print("the model discovered the reversal alignment on its own.")

```

Output:

```

T=13 reversal WITH attention: token acc 1.000 whole-seq 1.000
(the no-attention seq2seq of Listing 6 scored 0.522 / 0.000 here)

attention matrix (rows = output step, cols = source position, x10):
 0 0 0 0 0 0 0 0 0 0 0 0 0 10
 0 0 0 0 0 0 0 0 0 0 0 0 10 0
 0 0 0 0 0 0 0 0 0 0 0 10 0 0
 0 0 0 0 0 0 0 0 0 0 10 0 0 0
 0 0 0 0 0 0 0 0 0 10 0 0 0 0
 0 0 0 0 0 0 0 10 0 0 0 0 0 0
 0 0 0 0 0 0 10 0 0 0 0 0 0 0
 0 0 0 0 10 0 0 0 0 0 0 0 0 0
 0 0 0 10 0 0 0 0 0 0 0 0 0 0

```

```

0 0 10 0 0 0 0 0 0 0 0 0 0
0 10 0 0 0 0 0 0 0 0 0 0 0
10 0 0 0 0 0 0 0 0 0 0 0 0

```

mass concentrates on the ANTI-DIAGONAL: output t attends source $T-1-t$ --
the model discovered the reversal alignment on its own.

From 0.000 to 1.000 whole-sequence accuracy by letting the decoder look back at every encoder state, and the alignment matrix shows it looking at exactly the right one. The x4 score sharpening in the code earned its comment: without it the softmax stays uniform and attention never trains — the mirror image of the transformer's divide-by-sqrt(d).

Pitfalls, comparisons and practical tips

Cell selector:

Cell	States	Gates	Params (per H)	Long-lag memory	When
Vanilla RNN	h	none	1 block	tens of steps at best	toy tasks, theory
GRU	h	update, reset	3 blocks	good, weaker at extremes	latency/param budgets
LSTM	h and c	forget, input, output	4 blocks	best of the three	long dependencies
Transformer	none (attention)	—	(Ch. 16)	unlimited within window	default at scale

The recurring pitfalls:

- **Forgetting that BPTT accumulates.** Shared weights mean each time step ADDS into the same gradient buffers; assigning instead of accumulating is the classic scratch-implementation bug (Listing 1's `+=`).
- **Training RNNs without gradient clipping.** One exploding batch (Listing 2's $1.3\times$ regime) NaNs the run; global-norm clipping is not optional here.
- **Forget-gate bias at 0 or 1.** $f \sim 0.5-0.73$ decays memory as f^T ; initialize toward "remember" (bias 1-3; Listing 3 fails its own task without it).
- **Bidirectional layers in generative or streaming models.** The backward RNN needs the future; using BiLSTM features for next-token prediction is target leakage (Chapter 9's sin, sequence edition).
- **Evaluating with teacher forcing.** Feeding gold prefixes at eval time overstates quality by exactly the exposure-bias gap (15 points in Listing 6); always report free-running metrics.
- **Reading attention weights as explanations.** The anti-diagonal in Listing 7 is compelling because the task's true alignment is known; in general attention weights are one plausible attribution, not ground truth (Chapter 11's faithfulness caveat).

- **Uniform attention that never sharpens.** Weak score scale $\rightarrow$ flat softmax $\rightarrow$ weak gradients $\rightarrow$ flat softmax; check attention entropy early in training, fix with score scaling/temperature (the $\sqrt{d}$ lesson inverted).
- **Padding pollution.** Batched variable-length sequences need masking in the loss AND in attention scores; averaging loss over pad tokens silently deflates it.
- **State carryover bugs.** Truncated BPTT carries h (detached) between chunks; resetting it every batch erases context, never detaching it makes graphs grow unboundedly.

Interview questions and answers

Q1. Write the vanilla RNN update and explain what weight-tying across time buys and costs.

$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$, same weights at every step. Buys: constant parameter count for any sequence length, generalization across positions, and the ability to process variable lengths. Costs: BPTT multiplies by the SAME W_h at every step, so gradient decay/growth compounds as $\sim \lambda_{\max}^T$ with no averaging across layers (Listing 2), and each step's gradient must be accumulated into shared buffers during BPTT.

Q2. Explain BPTT step by step. How does it differ from ordinary backprop?

Unroll the recurrence into a T -layer graph, forward-cache every h_t , then walk backward: at step t , mask dh through the activation ($dz = dh \cdot (1 - h_t^2)$ for $\tanh$), accumulate $dW_x += x_t^T dz$, $dW_h += h_{t-1}^T dz$, and pass $dh = dz W_h^T$ to step $t-1$; add each step's readout gradient where outputs exist. Differences from plain backprop: gradient ACCUMULATION into shared weights, and memory scaling with T (all states cached) — hence truncated BPTT, which unrolls k steps per update, carrying state forward but cutting credit assignment beyond k .

Q3. Why do vanilla RNNs fail at long dependencies? Give the quantitative argument.

The Jacobian of one step is $\text{diag}(1-h^2) W_h^T$; through T steps the backward signal is a product of T such factors. If the typical spectral norm is ρ , the gradient reaching step 1 scales like ρ^T : $\rho < 1$ vanishes (no learning signal for long-lag credit), $\rho > 1$ explodes (NaNs). Listing 2: $1.2e-5$ at $T=100$ for healthy init; $40\times$ growth when 30% hotter. Tied weights make it worse than deep feedforward nets — no chance that independent layers cancel.

Q4. Walk through the LSTM's three gates and the two state-update equations.

From $z = [x_t, h_{t-1}]$: forget $f = \text{sigmoid}(W_f z + b_f)$ — how much of each memory slot survives; input $i = \text{sigmoid}(W_i z + b_i)$ — how much of candidate $g = \tanh(W_g z + b_g)$ is written; output $o = \text{sigmoid}(W_o z + b_o)$ — how much memory is revealed. Updates: $c_t = f \cdot c_{t-1} + i \cdot g$ (additive memory), $h_t = o \cdot \tanh(c_t)$ (gated readout). The additive c -path is the design's entire purpose: memory changes by gated increments, not by repeated nonlinear transformation.

Q5. Precisely how does the LSTM solve vanishing gradients?

The memory's backward recurrence is $dc_{t-1} = dc_t * f_t$ — elementwise multiplication by the forget gate, with no weight matrix and no activation derivative in the path (Listing 3's backward: $dc = dc \cdot f$). With f near 1 the gradient travels back nearly undamped, an identity highway like ResNet's skip (Chapter 14) — the "constant error carousel." Caveat that wins points: the protection is conditional on f being near 1; with textbook bias 1, $f \sim 0.73$ and $0.73^{40} \sim 3e-6$ — Listing 3's LSTM fails $T=40$ until the forget bias is raised to 3 ($f \sim 0.95$).

Q6. GRU vs LSTM: equations, parameter count, and when each wins.

GRU: update $u = \text{sigmoid}(\dots)$, reset $r = \text{sigmoid}(\dots)$, candidate $g = \tanh(W[x, r \cdot h])$, $h_t = u \cdot h_{t-1} + (1-u) \cdot g$. Two gates, one state, 3 weight blocks vs LSTM's 4 — 25% fewer parameters (Listing 4: 912 vs 1216). u couples forget and input (keep implies don't write); no output gate means the state is exposed every step. Empirically GRU $\sim$ LSTM on most tasks and cheaper; LSTM tends to win the longest dependencies — Listing 4's miniature: tied at $T=20$ (0.993), GRU 0.552 vs LSTM 0.840 at $T=40$, plausibly because the LSTM's cell is shielded behind its output gate while the GRU's memory is also its output.

Q7. What does the reset gate do that the update gate doesn't?

The update gate blends old state with new candidate AFTER the candidate is computed; the reset gate acts BEFORE, controlling how much of h_{t-1} feeds the candidate: $g = \tanh(W[x, r \cdot h])$. With $r \sim 0$ the candidate is computed from the current input alone — "start a fresh phrase" — while u can still decide to keep the old state in the blend. Reset = what the proposal may condition on; update = how much of the proposal is accepted.

Q8. When are bidirectional RNNs appropriate, and when are they an error?

Appropriate when the full sequence exists before outputs are needed and labels depend on right context: tagging (POS/NER), span extraction, acoustic frames, encoders feeding another model (ELMo; BERT is the attention version of the same idea). An error for autoregressive generation — the backward direction reads the tokens being predicted (leakage; Listing 5's task is chance forward, perfect bidirectionally for exactly this reason) — and impractical for streaming, where waiting for the sequence end costs full-sequence latency.

Q9. Describe the seq2seq architecture and its information bottleneck.

Encoder RNN consumes the source and passes its final hidden state to a decoder RNN that generates target tokens autoregressively until EOS. The decoder's ONLY source view is that single H-dimensional vector, so per-token capacity shrinks with source length: Listing 6, same model, token accuracy 0.999/0.860/0.522 at T=5/9/13, whole-sequence 0.000 by T=13 — the miniature of the BLEU-vs-length collapse that motivated attention. Fixes in order: reverse the source (shortens early dependencies), bigger H (delays it), attention (removes it).

Q10. What is teacher forcing, why is it used, and what problem does it create?

During training the decoder's input at step t is the GROUND-TRUTH token $t-1$, not the model's own prediction. Why: every step then trains on a clean prefix, the loss decomposes into independent per-step cross-entropies, gradients are stable, and (in transformers) all steps compute in parallel. The created problem is exposure bias: the model never practices recovering from its own mistakes — Listing 6 measures the same model at 0.855 (teacher-forced eval) vs 0.707 (free-running), a 15-point train/test mismatch. Mitigations: scheduled sampling, beam search, sequence-level losses; and note that large LMs still train teacher-forced.

Q11. Define exposure bias precisely and distinguish it from ordinary overfitting.

Exposure bias is a train/inference INPUT-distribution mismatch: training conditions on gold prefixes $p(y_t | y_{<t}, x)$; inference conditions on generated prefixes $p(y_t | \hat{y}_{<t}, x)$. One early error shifts the conditioning distribution off-manifold and errors compound with length. It is not overfitting: it exists at any training-set size and shows up as the gap between teacher-forced and free-running metrics on the SAME data (Listing 6's 15 points), not as a train/test generalization gap.

Q12. Write Bahdanau attention's equations and walk the computation.

Scores $e_{\{t,i\}} = v^T \tanh(W_a h_i^{\text{enc}} + U_a h_{\{t-1\}}^{\text{dec}})$ for every source position i ; weights $\alpha_t = \text{softmax}(e_t)$; context $\text{ctx}_t = \sum_i \alpha_{\{t,i\}} h_i^{\text{enc}}$; decoder update $h_t^{\text{dec}} = \tanh(W_x E[y_{\{t-1\}}] + W_h h_{\{t-1\}}^{\text{dec}} + W_c \text{ctx}_t + b)$. The score MLP is a learned compatibility function between "what I'm about to produce" (decoder state) and "what each source position holds" (encoder states); the softmax turns compatibilities into a convex mixture — a soft, differentiable lookup (Listing 7 implements forward and backward including the softmax Jacobian).

Q13. Bahdanau vs Luong attention — the concrete differences.

Score function: Bahdanau is additive, $v^T \tanh(W_a h_{\text{enc}} + U_a h_{\text{dec}})$ — a one-hidden-layer MLP; Luong is multiplicative — dot product $h_{\text{dec}}^T h_{\text{enc}}$ or bilinear $h_{\text{dec}}^T W h_{\text{enc}}$. Query state: Bahdanau uses the PREVIOUS decoder state (attention feeds the update); Luong the CURRENT one (attention refines the output). Trade-offs: additive handles different dimensionalities and small- d regimes gracefully; multiplicative is one matmul — faster — and with the $1/\sqrt{d}$ scaling correction became the transformer's scaled dot-product attention.

Q14. Why did attention solve the bottleneck, mechanically?

Capacity: the decoder now reads a length-proportional memory (all T encoder states) instead of one vector — information available per output token no longer shrinks with source length. Gradients: attention creates a DIRECT backward path from output t to the relevant encoder position — one hop instead of T recurrent steps — so credit assignment for long-range dependencies stops vanishing. Listing 7: same task, 0.522/0.000 without, 1.000/1.000 with, and the learned anti-diagonal shows the direct paths being used.

Q15. Are attention weights explanations?

Cautiously. Listing 7's anti-diagonal is a true alignment because the task's correct alignment is known and unique. In general: attention shows where the model READ, not what caused its output — weights can be diffuse, multiple heads disagree, and "attention is not explanation" experiments construct different weight patterns yielding identical predictions. Treat them as one attribution signal to cross-check against perturbation methods (Chapter 11's faithfulness standard), not as ground truth.

Q16. Your seq2seq's attention stays uniform and the model ignores it. Diagnose.

Chicken-and-egg: near-uniform softmax → weak, averaged gradients through attention → scores never differentiate. Causes: score scale too small (softmax of near-equal logits), encoder states insufficiently distinctive early, or the decoder solving the task without attention (bypass path too strong). Fixes: score temperature/scaling (Listing 7 needed x4 before the anti-diagonal emerged), proper score-layer init, monitoring attention entropy during training. The transformer's $1/\sqrt{d}$ is the same knob turned the other way — dot products too LARGE saturate softmax and also kill gradients.

Q17. How do you batch variable-length sequences correctly?

Pad to the batch max (or bucket by length to minimize padding), then mask everywhere the pad could leak: exclude pad positions from the loss (divide by real-token counts, not T), mask attention scores to -inf at pad positions BEFORE softmax, and for RNNs either use packed sequences or ensure state updates at pad steps are neutralized. Symptoms of getting it wrong: loss deflated by easy pad predictions, attention mass on padding, and length-correlated quality artifacts.

Q18. What is truncated BPTT and what does the truncation cost?

Process a long sequence in chunks of k steps: forward carries the hidden state across chunk boundaries (detached from the graph), backward runs only within the chunk. Memory and compute per update become $O(k)$ instead of $O(T)$. Cost: no gradient flows across boundaries, so dependencies longer than k receive no direct credit — the model can still USE longer context at inference (state carries it) but cannot LEARN to exploit patterns longer than k from gradients. Choosing k = expected dependency length is the practical compromise.

Q19. Compare RNN and transformer costs for sequence length n and width d .

RNN: $O(n \cdot d^2)$ compute, $O(1)$ state memory at inference, but SEQUENTIAL — n dependent steps, no parallelism across time; path length between positions i and j is $|i-j|$ (gradient hops). Transformer: $O(n^2 \cdot d)$ attention compute, fully parallel across positions at training, path length 1 between any pair. RNNs win memory/streaming latency; transformers win training throughput and long-range credit assignment — the trade that decided the field (Chapter 16), now partially revisited by state-space models.

Q20. Implement (in words) the backward pass through softmax attention weights.

Given $dctx$ (gradient at the context vector): $d\alpha_i = h_i^{enc} \cdot dctx$ (context is $\sum \alpha_i h_i$); $denc_i += \alpha_i \cdot dctx$. Through the softmax: $dscore = \alpha \cdot (d\alpha - (\alpha \cdot d\alpha))$ — the Jacobian of softmax contracts to "my gradient minus the α -weighted mean," which preserves the sum-to-one constraint. Then through the score function to its inputs (encoder states and decoder query). Listing 7 gradient-checks every one of these paths to $\sim 1e-8$; the softmax Jacobian line is where most hand-rolled implementations break.

Q21. Why initialize the LSTM forget-gate bias positive, and how positive?

At init, gates sit near $\text{sigmoid}(0) = 0.5$; memory then decays as 0.5^T — a length-20 dependency is attenuated a millionfold before any learning happens, so gradients that would teach the gate to remember never arrive. Bias $b_f = 1$ ($f \sim 0.73$) is the classic fix; for genuinely long lags it's insufficient ($0.73^{40} \sim 3e-6$) — Listing 3 needed $b_f = 3$ ($f \sim 0.95$). Chrono-init formalizes it: set biases so gate timescales match expected dependency lengths, $\sim \log(T)$ sized biases.

Q22. A char-level RNN language model outputs repetitive loops at generation time. Name causes and fixes.

Causes: greedy decoding's argmax attractor cycles (the model re-enters the same state), exposure bias amplifying small errors into off-manifold prefixes, and overconfident low-entropy output distributions. Fixes: sampling with temperature, top-k/nucleus sampling (Chapter 22), repetition penalties, beam search with diversity terms; training-side — scheduled sampling, more data/regularization to soften overconfidence. Diagnostic: compare teacher-forced perplexity (fine) with generation quality (poor) — the gap localizes the problem to decoding/exposure, not modeling.

Q23. Where does the hidden state's "memory" actually live, and what limits it?

In the H -dimensional vector's position in state space: the recurrent map carves attractors and transients that encode history — Listing 3's RNN latches sign as a bistable attractor. Limits: capacity (H dimensions $\sim H$ -ish scalars of long-term storage; the fixed-vector bottleneck is this limit at the seq2seq scale), interference (every step's update perturbs all coordinates — the GRU's $T=40$ failure), and trainability (gradients to sculpt the dynamics vanish with lag). Gates address interference and trainability; attention sidesteps capacity by externalizing memory.

Q24. Design a model for streaming sensor anomaly tagging (label each timestep, decisions within 100ms). Justify against this chapter.

Causal (unidirectional) GRU or LSTM, 1-2 layers: streaming forbids bidirectionality (future unavailable) and disfavors transformers with growing attention windows (recompute/memory per step); RNN inference is $O(1)$ state and per-step compute — ideal latency. GRU for the tighter compute budget unless dependencies are very long (Listing 4's trade). Truncated BPTT for training on long streams, clipping mandatory, forget/update biases initialized toward remember, per-step readout with masked loss. If a short lookahead (e.g. 500ms) is acceptable, a small fixed-lag window buys some right-context — the engineering middle ground between causal and bidirectional.

Q25. Chapter 12 gradient-checked a feedforward net. What extra care does gradient-checking an LSTM need?

(1) Check the RECURRENT paths specifically — perturb W_h /gate weights, not just readout, over multi-step sequences so errors that only appear through time surface; (2) include dc's chain ($dc = dc \cdot f + \text{contributions}$) — the classic bug is dropping the additive cc-path or the $dc \cdot f$ carry; (3) double precision and moderate T (numerical error grows with path length); (4) fix all randomness. Listings 1/3/4/7 check W_h , gate blocks, and every attention path — worst errors $1e-10$ to $7e-6$, the LSTM/GRU ones larger simply because paths are longer.

Q26. Why does reversing the source sequence help vanilla seq2seq (Sutskever's trick)?

With the source reversed, the FIRST target tokens align with the LAST-read source tokens — the shortest possible recurrent path connects them, so early decoding (whose errors compound most) conditions on the freshest memory. Average dependency length is unchanged but the variance is redistributed to favor sentence-initial alignment, which empirically lifted BLEU substantially. It is a hack that diagnoses the disease: dependency length through the recurrence is the binding constraint — attention treats it properly by making every path length 1.

Q27. How would you decide between an RNN-with-attention and a transformer today?

Default transformer: parallel training, path-length-1 credit assignment, ecosystem/pretrained weights (Chapters 16, 20). Choose recurrent when: streaming with strict per-step latency and $O(1)$ memory; very long sequences where n^2 attention is prohibitive and windowing hurts; tiny deployment budgets; or small-data regimes where a compact GRU generalizes fine. Modern middle grounds worth naming: state-space models (Mamba-family) with recurrent $O(n)$ inference and parallel training, and hybrid attention+recurrence stacks — the pendulum currently swinging back toward recurrence for efficiency.

Q28. Trace this chapter's arc as answers to failure modes, LSTM to attention.

Vanilla RNN: state + tied weights → fails long lags because T Jacobians multiply (Listing 2's $1e-5$ at $T=100$). LSTM: additive gated memory → gradient path $dc = dc \cdot f$, near-identity when $f \sim 1$ (Listing 3: chance → 0.84 at $T=40$) — the recurrent ResNet. GRU: same idea, coupled gates, $3/4$ the parameters, slightly shorter reach (Listing 4). Bidirectionality: labels that live in the future (Listing 5: 0.48 → 1.00). Seq2seq: variable-length mapping, but one vector must carry the whole source (0.000 at $T=13$) and teacher forcing leaves a 15-point exposure gap (Listing 6). Attention: length-proportional memory with one-hop gradients (1.000/1.000, learned anti-diagonal, Listing 7) — and once attention does the remembering, the recurrence is dead weight; removing it is Chapter 16.